Analysis of Data Communication in a Web Application Using Browser Developer Tools

Case Study 5

Aim

To analyze how a web browser and a web server communicate by inspecting requests and responses with browser developer tools.

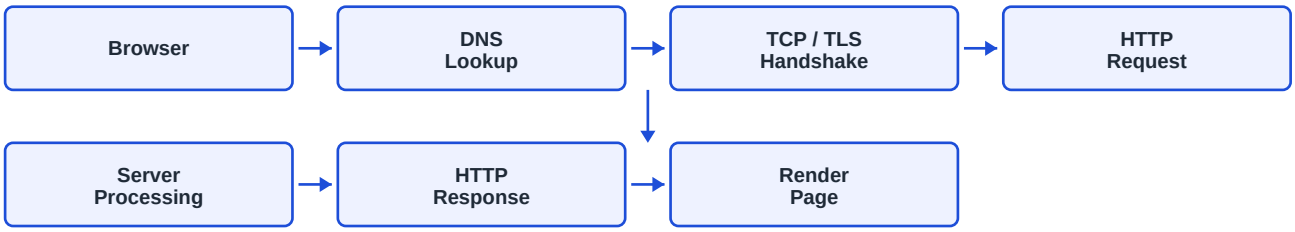
Objective

- Open and navigate the browser Developer Tools (DevTools).
- Capture and examine HTTP requests and responses in the Network tab.
- Understand methods, status codes, headers, cookies and payloads.
- Measure page load performance and identify slow resources.

Tools Used

Google Chrome (or Microsoft Edge / Firefox) DevTools – opened with **F12** or **Ctrl + Shift + I**. Any web application with a login or search form can be inspected (for example, a practice site such as *httpbin.org* or *reqres.in*).

Overview of Web Communication



Procedure

1. Open the web application in Chrome and press F12.
2. Select the **Network** tab and tick “Preserve log”.
3. Reload the page (Ctrl + R) and observe the list of requests.
4. Click the main document request and inspect the Headers, Preview, Response, Cookies and Timing sub-tabs.
5. Submit a form (login or search) and examine the POST / GET request and its payload.
6. Filter by type (Fetch/XHR, JS, CSS, Img) to isolate API calls.
7. Check the **Application** tab for cookies / local storage and the **Console** tab for errors.

Key Elements Examined

Element	What it shows	Example
Request URL	Resource being requested	https://example.com/api/login
Request Method	Action to perform	GET, POST, PUT, DELETE
Status Code	Result of the request	200 OK, 301 Redirect, 404 Not Found, 500 Server Error
Request Headers	Browser information and settings	User-Agent, Accept, Cookie, Authorization

Element	What it shows	Example
Response Headers	Server information and policies	Content-Type, Cache-Control, Set-Cookie
Payload	Data sent to the server	{"email":"user@mail.com","password":"****"}
Response	Data returned by the server	HTML page or JSON object
Timing	Waiting, download and total time	DNS, connect, TTFB, content download

HTTP Status Code Categories

Range	Meaning	Examples
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server error	500 Internal Server Error, 503 Service Unavailable

Sample Request / Response

```
POST /api/login HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 48

{"email":"user@mail.com","password":"********"}

HTTP/1.1 200 OK
Content-Type: application/json
Set-Cookie: session=abc123; HttpOnly; Secure

{"status":"success","user":"user@mail.com"}
```

The sample above is illustrative; actual headers and values depend on the application being inspected.

Observations

- The initial HTML document is fetched first; CSS, JavaScript and images are then requested in parallel.
- Form submission sends data as a POST request with a payload; page search typically uses GET with query parameters.
- Responses to API calls are in JSON, while page requests return HTML.
- Secure sites use HTTPS (TLS); sensitive cookies carry the HttpOnly and Secure flags.
- The Timing tab reveals bottlenecks such as large images or slow server response (high TTFB).

Performance and Security Insights

- Compressing images and enabling caching reduce load time.
- Passwords must never be sent over plain HTTP.
- A 401 / 403 status shows authentication or authorization failure; a 5xx status indicates a server-side issue.

Results

- HTTP requests, responses, headers, cookies and payloads were successfully captured and analyzed.
- The complete request–response cycle between browser and server was understood.

- Performance issues and security-related headers could be identified using DevTools.

Conclusion

Browser developer tools are powerful for understanding and debugging data communication in web applications. They help developers and testers verify API behaviour, troubleshoot errors, improve performance and check basic security practices without extra software.